

CG Lab Project3: 光线追踪 实验报告

袁晨圃 2023K8009929012

 Repo cauphenuny/ucas-graphics

一、实验环境以及运行效果

1.1 实验环境

- 编译器：支持 C++20 的编译器（GCC 10+ / Clang 12+ / MSVC 2019+）
- 构建工具：xmake
- 依赖库：
 - GLM：用于向量和矩阵运算（代码中已经包含）
 - OpenMP：用于多线程并行计算（编译器通常自带支持）
 - STB Image：用于图像读写（代码中已经包含）
 - meshark（来自 p2: mesh simplification），用于加载 wavefront obj 模型文件

1.2 编译与运行

```
1 # 编译项目
2 xmake
3
4 # 运行不同的场景
5 xmake run project3 0 final0.ppm # final scene 0
6 xmake run project3 1 final1.ppm # final scene 1
7
8 xmake run project3 2 cornell_box.ppm # cornell box
9 xmake run project3 2 cornell_box1.ppm --light
10 xmake run project3 2 cornell_box2.ppm --metal --light
11 xmake run project3 2 cornell_box3.ppm --glass --light
12
13 xmake run project3 3 earth_texture.ppm # earth texture
14 xmake run project3 4 perlin_noise.ppm # perlin noise
15 xmake run project3 5 basic_shapes.ppm # basic shapes
16 xmake run project3 6 mesh_rendering.ppm # mesh rendering
17 xmake run project3 7 lighting.ppm # lighting demo
```

1.3 运行效果

1.3.1 基础

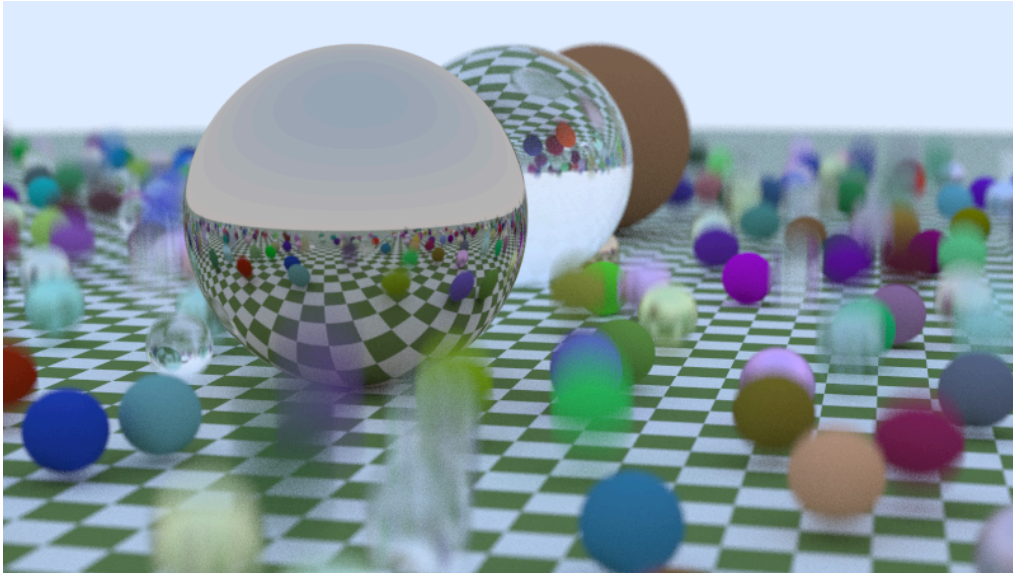


图 1 不同材质的球体渲染效果，包含景深效果和动态模糊

1.3.2 网格模型



图 2 复杂网格模型渲染 (p2 的 complex_bunny)

1.3.3 纹理与介质

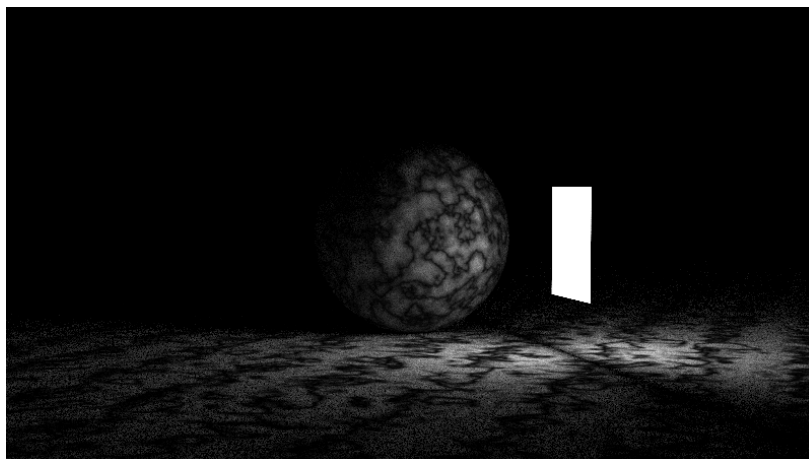


图 3 柏林噪声纹理和光源

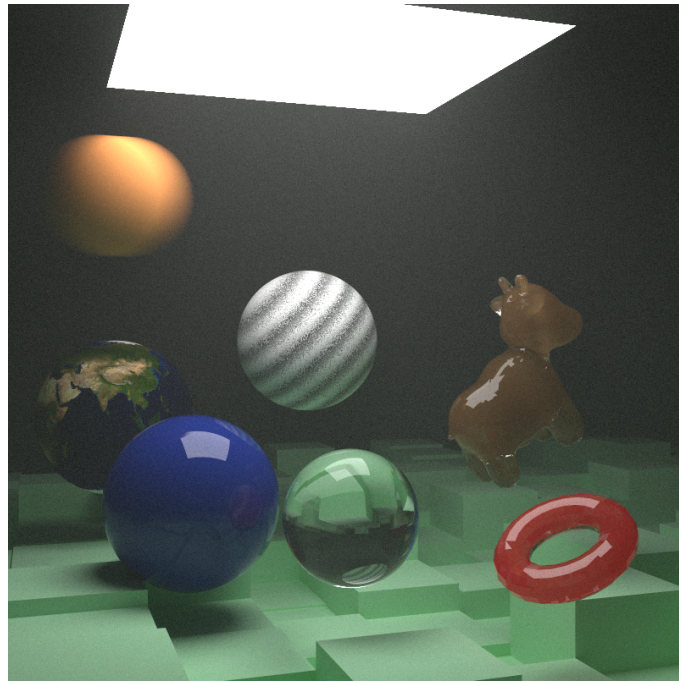


图 4 纹理贴图、体积雾、次表面散射

1.3.4 重要性采样

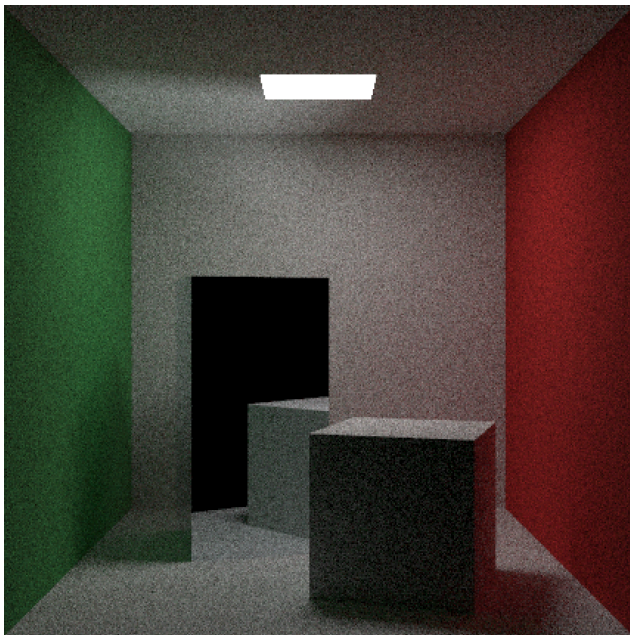


图 5 康奈尔盒，金属材质，1000 samples per pixel

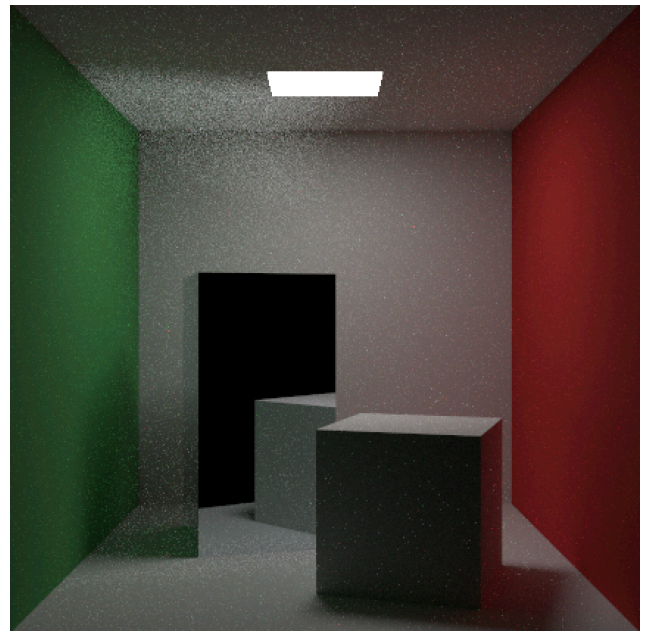


图 6 sample 数不变，采样光源加速收敛

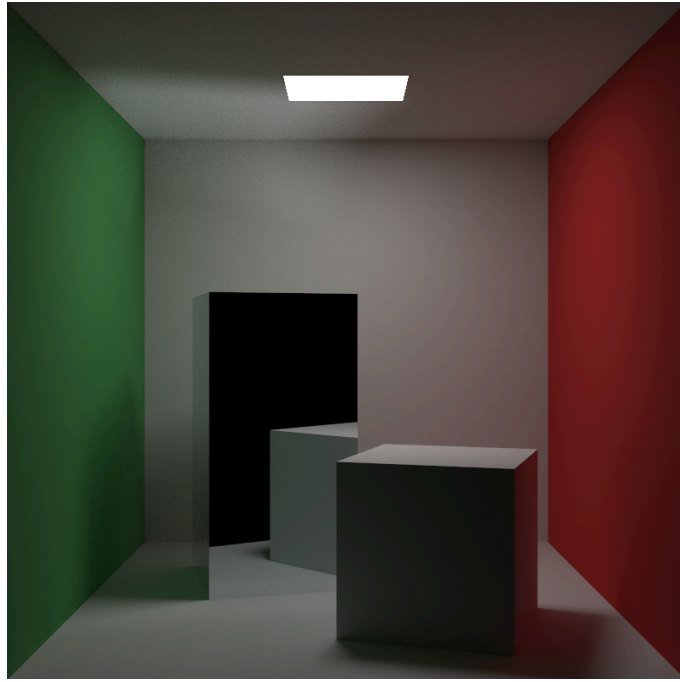


图 7 康奈尔盒，金属材质，20000 samples per pixel

1.3.5 光谱渲染

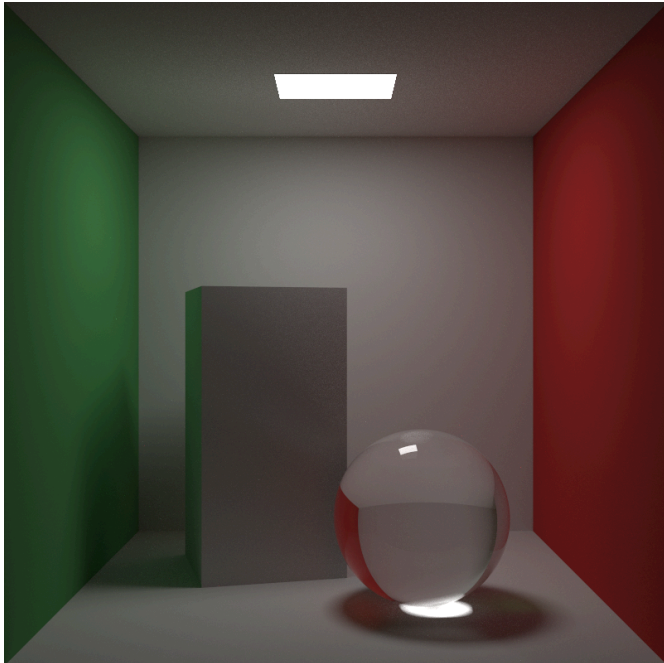


图 8 康奈尔盒，玻璃材质

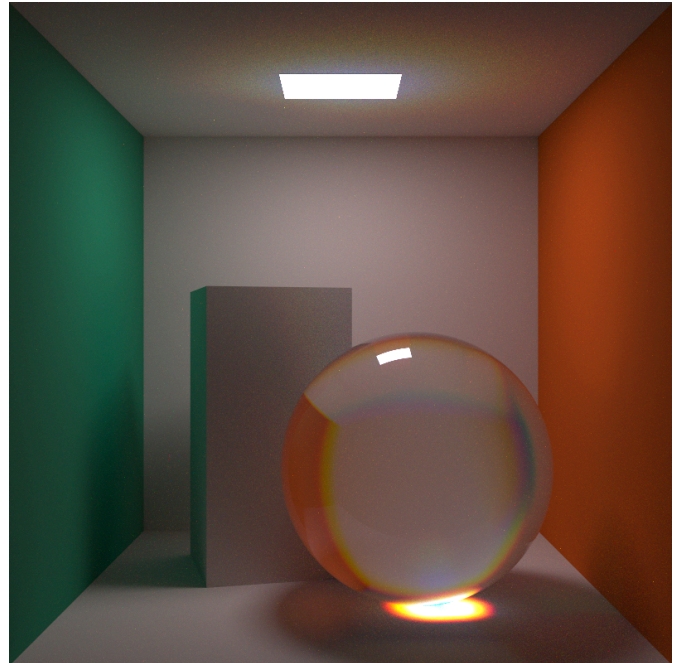


图 9 使用光谱渲染实现色散效果

渲染器支持多种场景和材质效果，包括：

- 漫反射材质 (Lambertian)
- 金属材质 (Metal)
- 电介质材质 (Dielectric, 如玻璃)
- 发光材质 (Emissive Light)
- 纹理映射和柏林噪声
- 景深效果 (Depth of Field)
- 运动模糊 (Motion Blur)
- 体积雾与次表面散射

基本跟着 RayTracing in One Weekend 系列教程完整地实现了一遍，外加一些其他特性，比如光谱渲染、网格加载、多线程等。

代码量还挺大的，由于时间关系报告接下来的内容可能含有一些 AI 辅助生成的东西

二、代码介绍

本章节详细介绍光线追踪项目的核心代码实现，从基础架构到高级特性，分为四个部分：基础路径追踪器、性能优化与复杂特性、蒙特卡洛积分与重要性采样、光谱渲染。

2.1 第一部分：基础架构与路径追踪器

2.1.1 基础数学：Vec3 向量运算

向量是光线追踪的基础数据结构，用于表示位置、方向、颜色等。我们的实现提供了完整的三维向量运算支持。

```
1 class Vec3 {cpp  
2     double d[3];  
3  
4 public:  
5     Vec3() = default;  
6     Vec3(double x, double y, double z) { d[0] = x, d[1] = y, d[2] = z; }  
7  
8     double x() const { return d[0]; }  
9     double y() const { return d[1]; }  
10    double z() const { return d[2]; }  
11  
12    // 基本运算符重载  
13    Vec3 operator-() const { return Vec3{-x(), -y(), -z()}; }  
14    Vec3& operator+=(const Vec3& v) {  
15        d[0] += v.x(); d[1] += v.y(); d[2] += v.z();  
16        return *this;  
17    }  
18  
19    // 向量模长  
20    double sqnorm() const { return x() * x() + y() * y() + z() * z(); }  
21    double norm() const { return std::sqrt(sqnorm()); }  
22    Vec3 normalized() const { return *this / norm(); }  
23 };
```

核心功能：

- 基本算术运算（加减乘除）
- 点积与叉积运算
- 向量归一化
- 随机向量生成（用于光线散射）

```
1 // 向量运算cpp  
2 inline double dot(const Vec3& u, const Vec3& v) {  
3     return u.x() * v.x() + u.y() * v.y() + u.z() * v.z();  
4 }  
5  
6 inline Vec3 cross(const Vec3& u, const Vec3& v) {  
7     return Vec3(  
8         u.y() * v.z() - u.z() * v.y(),  
9         u.z() * v.x() - u.x() * v.z(),  
10        u.x() * v.y() - u.y() * v.x()  
11    );  
12 }
```

```

13
14 // 反射与折射
15 inline Vec3 reflect(const Vec3& v, const Vec3& n) {
16     return v - 2 * dot(v, n) * n;
17 }
18
19 inline Vec3 refract(const Vec3& uv, const Vec3& n, double etai_over_etat) {
20     auto cos_theta = std::fmin(dot(-uv, n), 1.0);
21     Vec3 r_out_perp = etai_over_etat * (uv + cos_theta * n);
22     Vec3 r_out_parallel = -std::sqrt(std::fabs(1.0 - r_out_perp.sqrnorm())) * n;
23     return r_out_perp + r_out_parallel;
24 }

```

随机采样功能:

```

1 // 单位球面上的随机点 (重要性采样)
2 static Vec3 random_unit() {
3     while (true) {
4         auto p = Vec3::random(-1, 1);
5         auto sqrnorm = p.sqrnorm();
6         if (1e-10 < sqrnorm && sqrnorm < 1) {
7             return p * (1.0 / std::sqrt(sqrnorm));
8         }
9     }
10 }
11
12 // 余弦加权采样 (用于漫反射)
13 static Vec3 random_cosine_z() {
14     auto r1 = random_double();
15     auto r2 = random_double();
16     auto phi = 2 * pi * r1;
17     auto x = std::cos(phi) * std::sqrt(r2);
18     auto y = std::sin(phi) * std::sqrt(r2);
19     auto z = std::sqrt(1 - r2);
20     return Vec3(x, y, z);
21 }

```

cpp

2.1.2 光线与相机

2.1.2.1 Ray: 光线类

光线是路径追踪的核心概念，由起点、方向和时间组成。

```

1 class Ray {
2     Point3 orig;
3     Vec3 dir;
4     double tm;
5
6 public:
7     Ray(const Point3& origin, const Vec3& direction, double time = 0)
8         : orig(origin), dir(direction), tm(time) {}
9
10    const Point3& origin() const { return orig; }
11    const Vec3& direction() const { return dir; }
12    Point3 at(double t) const { return orig + dir * t; }
13    double time() const { return tm; }

```

cpp

```
14 };
```

光线的参数方程： $P(t) = O + tD$ ，其中 O 是起点， D 是方向， t 是参数。

2.1.2.2 Camera: 相机类

相机负责生成视角光线，支持视场角、景深、散焦模糊等特性。

```
1 struct Camera {
2     double aspect_ratio = 1.0;
3     int image_width = 100;
4     int samples_per_pixel = 10;
5     int max_depth = 10;
6     Color background = Color(0.7, 0.8, 1);
7
8     double vfov = 90.0;           // 垂直视场角
9     Point3 lookfrom = Point3(0, 0, 0);
10    Point3 lookat = Point3(0, 0, -1);
11    Vec3 vup = Vec3(0, 1, 0);
12
13    double defocus_angle = 0;       // 散焦角度
14    double focus_dist = 10;        // 焦距
15 };
```

相机初始化:

```
1 void initialize() {
2     image_height = std::max(static_cast<int>(image_width / aspect_ratio), 1);
3     center = lookfrom;
4
5     // 计算视口尺寸
6     auto theta = degrees_to_radians(vfov);
7     auto h = std::tan(theta / 2);
8     auto viewport_height = 2.0 * focus_dist * h;
9     auto viewport_width = ratio * viewport_height;
10
11    // 建立相机坐标系
12    w = (lookfrom - lookat).normalized(); // 向后
13    u = cross(vup, w).normalized();       // 向右
14    v = cross(w, u);                      // 向上
15
16    // 计算像素间距
17    pixel_delta_u = viewport_u / image_width;
18    pixel_delta_v = viewport_v / image_height;
19 }
```

2.1.3 几何形状

2.1.3.1 Hittable 抽象基类

所有几何体都继承自 Hittable 接口，提供光线相交测试和包围盒计算。

```
1 struct HitResult {
2     Point3 p;           // 交点位置
3     Vec3 normal;        // 表面法线
4     double t;           // 光线参数
5     bool front_face;     // 是否为正面
```

```

6     std::shared_ptr<Material> mat;
7     double u, v;           // 纹理坐标
8
9     void set_face_normal(const Ray& r, const Vec3& outward_normal) {
10         front_face = dot(r.direction(), outward_normal) < 0;
11         normal = front_face ? outward_normal : -outward_normal;
12     }
13 };
14
15 class Hittable {
16 public:
17     virtual ~Hittable() = default;
18     virtual bool hit(const Ray& ray, Interval interval, HitResult& result) const = 0;
19     virtual BoundingBox bounding_box() const = 0;
20 };

```

2.1.3.2 Sphere: 球体

球体是最基础的几何形状，支持静态和运动两种模式。

```

1 class Sphere : public Hittable {
2     double radius;
3     Ray center; // 支持运动：起点和方向定义运动轨迹
4     std::shared_ptr<Material> mat;
5     BoundingBox bbox;
6
7 public:
8     // 运动球体构造函数
9     Sphere(const Point3& center_start, const Point3& center_end,
10           double radius, std::shared_ptr<Material> mat)
11         : center(center_start, center_end - center_start),
12           radius(std::max(0., radius)), mat(mat) {
13         // 计算动态包围盒
14         auto rvec = Vec3(radius, radius, radius);
15         auto box0 = BoundingBox::diag(center.at(0) - rvec, center.at(0) + rvec);
16         auto box1 = BoundingBox::diag(center.at(1) - rvec, center.at(1) + rvec);
17         bbox = BoundingBox::combine(box0, box1);
18     }
19 };

```

球体光线相交：

求解方程： $\|P(t) - C\|^2 = r^2$

```

1 bool hit(const Ray& ray, Interval interval, HitResult& result) const override {
2     auto current_center = center.at(ray.time());
3     auto oc = current_center - ray.origin();
4     auto a = ray.direction().sqnorm();
5     auto h = dot(ray.direction(), oc);
6     auto c = oc.sqnorm() - radius * radius;
7     auto discriminant = h * h - a * c;
8
9     if (discriminant < 0) return false;
10
11     auto sqrt_d = std::sqrt(discriminant);
12     auto root = (h - sqrt_d) / a;
13     if (!interval.surrounds(root)) {
14         root = (h + sqrt_d) / a;
15         if (!interval.surrounds(root)) return false;
16     }

```

```

17
18     result.t = root;
19     result.p = ray.at(root);
20     Vec3 outward_normal = (result.p - current_center) / radius;
21     result.set_face_normal(ray, outward_normal);
22     result.mat = mat;
23     return true;
24 }

```

球面 UV 映射:

```

1  static auto sphere_uv(const Point3& p) {
2      // p: 单位球面上的点
3      // u: [0,1] 水平角度
4      // v: [0,1] 垂直角度
5      auto theta = std::acos(-p.y());
6      auto phi = std::atan2(-p.z(), p.x()) + pi;
7      auto u = phi / (2 * pi);
8      auto v = theta / pi;
9      return std::make_tuple(u, v);
10 }

```

2.1.4 基础材质

材质定义了光线与物体表面的交互方式，包括散射和发光。

```

1  struct ScatterResult {
2      Color attenuation; // 衰减系数
3      std::variant<std::unique_ptr<Vec3PDF>, Ray> scattered; // 散射光线或PDF
4  };
5
6  class Material {
7  public:
8      virtual bool scatter(const Ray& r_in, const HitResult& hit,
9                          ScatterResult& result) const { return false; }
10     virtual Color emit(const Ray& r_in, const HitResult& hit) const {
11         return Color(0, 0, 0);
12     }
13     virtual double scattering_pdf(const Ray& r_in, const HitResult& hit,
14                                 const Ray& scattered) const { return 1.0; }
15 };

```

2.1.4.1 Lambertian: 漫反射材质

理想漫反射表面，反射光线均匀分布在半球面上，遵循余弦定律。

```

1  class Lambertian : public Material {
2      std::shared_ptr<Texture> tex;
3
4  public:
5      Lambertian(const Color& albedo)
6          : tex(std::make_shared<ColorTexture>(albedo)) {}
7
8      bool scatter(const Ray& r_in, const HitResult& hit,
9                  ScatterResult& result) const override {
10         result.scattered = std::make_unique<CosinePDF>(hit.normal);

```

```

11     result.attenuation = tex->value(hit.u, hit.v, hit.p);
12     return true;
13 }
14
15 double scattering_pdf(const Ray& r_in, const HitResult& hit,
16                     const Ray& scattered) const override {
17     auto cosine = dot(hit.normal, scattered.direction().normalized());
18     return (cosine < 0) ? 0.0 : (cosine / pi);
19 }
20 };

```

散射概率密度函数: $p(\omega) = \frac{\cos \theta}{\pi}$

2.1.4.2 Metal: 金属材质

完美镜面反射, 支持模糊参数 (fuzz) 模拟粗糙表面。

```

1  class Metal : public Material {
2      Color albedo;
3      double fuzz; // 模糊系数
4
5  public:
6      Metal(const Color& albedo, double fuzz = 0) : albedo(albedo), fuzz(fuzz) {}
7
8      bool scatter(const Ray& r_in, const HitResult& hit,
9                  ScatterResult& result) const override {
10         Vec3 reflected = reflect(r_in.direction().normalized(), hit.normal)
11                             .normalized() + (fuzz * Vec3::random_unit());
12         result.scattered = Ray(hit.p, reflected, r_in.time());
13         result.attenuation = albedo;
14         return dot(reflected, hit.normal) > 0;
15     }
16 };

```

反射方向: $R = D - 2(D \cdot N)N$

2.1.4.3 Dielectric: 电介质材质

透明材质, 支持折射和反射, 使用 Schlick 近似计算菲涅尔反射率。

```

1  class Dielectric : public Material {
2      double refraction_index;
3
4  public:
5      Dielectric(double ri) : refraction_index(ri) {}
6
7      bool scatter(const Ray& r_in, const HitResult& hit,
8                  ScatterResult& result) const override {
9         result.attenuation = Color(1.0, 1.0, 1.0);
10         double etai_over_etat = hit.front_face ?
11             (1.0 / refraction_index) : refraction_index;
12
13         Vec3 unit_direction = r_in.direction().normalized();
14         double cos_theta = std::fmin(dot(-unit_direction, hit.normal), 1.0);
15         double sin_theta = std::sqrt(1.0 - cos_theta * cos_theta);
16
17         bool cannot_refract = etai_over_etat * sin_theta > 1.0;
18         Vec3 direction;
19
20         if (cannot_refract || reflectance(cos_theta, etai_over_etat) > random_double()) {
21             direction = reflect(unit_direction, hit.normal);

```

```

22     } else {
23         direction = refract(unit_direction, hit.normal, etai_over_etat);
24     }
25
26     result.scattered = Ray(hit.p, direction, r_in.time());
27     return true;
28 }
29
30 static double reflectance(double cosine, double ref_idx) {
31     // Schlick 近似
32     auto r0 = (1 - ref_idx) / (1 + ref_idx);
33     r0 = r0 * r0;
34     return r0 + (1 - r0) * std::pow((1 - cosine), 5);
35 }
36 };

```

2.1.5 渲染管线

2.1.5.1 递归路径追踪

核心渲染函数采用递归方式追踪光线，直到达到最大深度或击中光源。

```

1  auto ray_color(const Ray& ray, const Hittable& world,                                     cpp
2      Samplable* sample_target, int depth) -> Color const {
3      if (depth <= 0) return Color(0, 0, 0); // 深度限制
4
5      HitResult hit_result;
6      if (!world.hit(ray, Interval(0.001, infinity), hit_result)) {
7          return background; // 未击中物体，返回背景色
8      }
9
10     ScatterResult scatter_result;
11     Color color_emit = hit_result.mat->emit(ray, hit_result);
12
13     if (!hit_result.mat->scatter(ray, hit_result, scatter_result)) {
14         return color_emit; // 不散射，仅返回自发光
15     }
16
17     auto& [attenuation, scattered] = scatter_result;
18
19     // 处理散射：使用模式匹配处理不同类型的散射结果
20     auto color_scatter = Match{std::move(scattered)}{
21         [&](Ray ray) -> Color {
22             // 镜面反射/折射：直接使用散射光线
23             return attenuation * ray_color(ray, world, sample_target, depth - 1);
24         },
25         [&](std::unique_ptr<Vec3PDF> pdf) -> Color {
26             // 漫反射：使用概率密度函数进行重要性采样
27             std::unique_ptr<Vec3PDF> scatter_pdf =
28                 sample_target ? std::make_unique<MixturePDF>(
29                     std::make_unique<ObjectPDF>(*sample_target, hit_result.p),
30                     std::move(pdf))
31                     : std::move(pdf);
32             auto scattered_ray = Ray(hit_result.p, scatter_pdf->generate(), ray.time());
33             double sampling_prob = scatter_pdf->value(scattered_ray.direction());
34             double scatter_prob = hit_result.mat->scattering_pdf(ray, hit_result, scattered_ray);

```

```

35     Color sampled_color = ray_color(scattered_ray, world, sample_target, depth - 1);
36     return (attenuation * scatter_prob * sampled_color) / sampling_prob;
37 };
38
39     return color_emit + color_scatter;
40 }

```

渲染方程: $L_o = L_e + \int f_r L_i \cos \theta d\omega$

2.1.5.2 Gamma 校正与抗锯齿

```

1  class Color : public Vec3 {
2  public:
3      Color to_gamma() {
4          return Color(std::sqrt(r()), std::sqrt(g()), std::sqrt(b()));
5      }
6
7      std::tuple<uint8_t, uint8_t, uint8_t> to_byte() const {
8          auto red = std::clamp(r(), 0.0, 0.9999);
9          auto green = std::clamp(g(), 0.0, 0.9999);
10         auto blue = std::clamp(b(), 0.0, 0.9999);
11         return std::make_tuple(int(256 * red), int(256 * green), int(256 * blue));
12     }
13 };

```

分层采样 (Stratified Sampling):

```

1  auto sample_square_stratified(int si, int sj) const {
2      auto px = ((si + random_double()) * recip_sqrt_spp) - 0.5;
3      auto py = ((sj + random_double()) * recip_sqrt_spp) - 0.5;
4      return Vec3{px, py, 0.0};
5  }
6
7  // 主渲染循环
8  for (int sj = 0; sj < sqrt_spp; sj++) {
9      for (int si = 0; si < sqrt_spp; si++) {
10         Ray r = get_ray(i, j, si, sj);
11         pixel_color += ray_color(r, world, sample_target, max_depth);
12     }
13 }

```

2.1.6 散焦模糊 (景深)

模拟真实相机的光圈效果, 通过在散焦盘上随机采样光线起点实现景深。

```

1  auto sample_defocus_disk() const {
2      auto p = Vec3::random_in_unit_disk();
3      return center + (p.x() * defocus_disk_u) + (p.y() * defocus_disk_v);
4  }
5
6  auto get_ray(int i, int j, int si, int sj) const -> Ray {
7      auto offset = sample_square_stratified(si, sj);
8      auto pixel_sample = pixel00_loc +
9          ((i + offset.x()) * pixel_delta_u) +
10         ((j + offset.y()) * pixel_delta_v);
11
12     auto ray_origin = (defocus_angle <= 0) ? center : sample_defocus_disk();

```

```

13     auto ray_time = random_double();
14     return Ray(ray_origin, pixel_sample - ray_origin, ray_time);
15 }

```

散焦半径计算:

```

1 auto defocus_radius = focus_dist * std::tan(degrees_to_radians(defocus_angle) / 2);
2 defocus_disk_u = defocus_radius * u;
3 defocus_disk_v = defocus_radius * v;

```

2.2 第二部分：性能优化与复杂特性

2.2.1 运动模糊

通过为每条光线分配随机时间戳，并在该时刻采样运动物体的位置，实现运动模糊效果。

2.2.1.1 MovingSphere: 运动球体

```

1 class Sphere : public Hittable {
2     Ray center; // 用 Ray 存储运动轨迹
3
4 public:
5     Sphere(const Point3& center_start, const Point3& center_end,
6            double radius, std::shared_ptr<Material> mat)
7         : center(center_start, center_end - center_start),
8           radius(std::max(0., radius)), mat(mat) {
9         // 合并两个时刻的包围盒
10        auto box0 = BoundingBox::diag(center.at(0) - rvec, center.at(0) + rvec);
11        auto box1 = BoundingBox::diag(center.at(1) - rvec, center.at(1) + rvec);
12        bbox = BoundingBox::combine(box0, box1);
13    }
14
15    bool hit(const Ray& ray, Interval interval, HitResult& result) const override {
16        auto current_center = center.at(ray.time()); // 在光线时间戳时的位置
17        // ... 相交测试
18    }
19 };

```

关键思想:

- 光线携带时间信息 `ray.time()`
- 物体位置根据时间插值: $C(t) = C_0 + t(C_1 - C_0)$
- 包围盒覆盖整个运动范围

2.2.2 BVH 加速结构

层次包围盒 (Bounding Volume Hierarchy) 通过空间划分大幅减少光线-物体相交测试次数。

2.2.2.1 AABB: 轴对齐包围盒

```

1 class BoundingBox {
2 public:
3     Interval x, y, z;
4

```

```

5     static BoundingBox diag(const Point3& a, const Point3& b) {
6         auto x = Interval(std::fmin(a.x(), b.x()), std::fmax(a.x(), b.x()));
7         auto y = Interval(std::fmin(a.y(), b.y()), std::fmax(a.y(), b.y()));
8         auto z = Interval(std::fmin(a.z(), b.z()), std::fmax(a.z(), b.z()));
9         return BoundingBox(x, y, z);
10    }
11
12    static BoundingBox combine(const BoundingBox& box0, const BoundingBox& box1) {
13        auto x = Interval::combine(box0.x, box1.x);
14        auto y = Interval::combine(box0.y, box1.y);
15        auto z = Interval::combine(box0.z, box1.z);
16        return BoundingBox(x, y, z);
17    }
18
19    bool hit(const Ray& ray, Interval interval) const {
20        for (int axis = 0; axis < 3; axis++) {
21            const Interval& ax = axis_interval(axis);
22            const double inv_d = 1.0 / ray.direction()[axis];
23
24            auto t0 = (ax.min - ray.origin()[axis]) * inv_d;
25            auto t1 = (ax.max - ray.origin()[axis]) * inv_d;
26
27            if (t0 > t1) std::swap(t0, t1);
28
29            interval.min = std::fmax(t0, interval.min);
30            interval.max = std::fmin(t1, interval.max);
31
32            if (interval.max <= interval.min) return false;
33        }
34        return true;
35    }
36 };

```

2.2.2.2 BVHNode: BVH 节点

```

1  class BVHNode : public Hittable {
2      std::shared_ptr<Hittable> left, right;
3      BoundingBox bbox;
4
5  public:
6      BVHNode(std::vector<std::shared_ptr<Hittable>> objects,
7              size_t start, size_t end) {
8          // 计算包围盒
9          bbox = BoundingBox::empty();
10         for (size_t i = start; i < end; i++) {
11             bbox = BoundingBox::combine(bbox, objects[i]->bounding_box());
12         }
13
14         int axis = bbox.longest_axis(); // 选择最长轴分割
15
16         size_t object_span = end - start;
17         if (object_span == 1) {
18             left = right = objects[start];
19         } else if (object_span == 2) {
20             left = objects[start];
21             right = objects[start + 1];
22         } else {
23             std::sort(objects.begin() + start, objects.begin() + end,
24                       comparator[axis]);

```

cpp

```

25     auto mid = start + object_span / 2;
26     left = BVHNode::create(objects, start, mid);
27     right = BVHNode::create(objects, mid, end);
28 }
29 }
30
31 bool hit(const Ray& ray, Interval interval, HitResult& result) const override {
32     if (!bbox.hit(ray, interval)) return false; // 早期剔除
33
34     bool hit_left = left->hit(ray, interval, result);
35     bool hit_right = right->hit(ray,
36         hit_left ? Interval(interval.min, result.t) : interval, result);
37
38     return hit_left || hit_right;
39 }
40 };

```

BVH 构建策略:

1. 计算所有物体的总包围盒
2. 选择最长轴进行分割
3. 排序物体并递归构建子树
4. 叶节点包含 1-2 个物体

时间复杂度: $O(\log N)$ 查询, $O(N \log N)$ 构建

2.2.3 纹理映射

纹理系统支持纯色、图像、程序化噪声等多种类型。

2.2.3.1 Texture 基类

```

1 class Texture {
2 public:
3     virtual ~Texture() = default;
4     virtual Color value(double u, double v, const Point3& p) const = 0;
5 };

```

cpp

2.2.3.2 ColorTexture: 纯色纹理

```

1 class ColorTexture : public Texture {
2     Color albedo;
3
4 public:
5     ColorTexture(const Color& albedo) : albedo(albedo) {}
6
7     Color value(double u, double v, const Point3& p) const override {
8         return albedo;
9     }
10 };

```

cpp

2.2.3.3 ImageTexture: 图像纹理

```

1 class ImageTexture : public Texture {
2     Image image;
3
4 public:
5     ImageTexture(const char* filename) : image(filename) {}

```

cpp

```

6
7     Color value(double u, double v, const Point3& p) const override {
8         if (image.width() == 0 || image.height() == 0) {
9             return Color(0.0, 1.0, 1.0); // cyan for debug
10        }
11
12        u = std::clamp(u, 0.0, 1.0);
13        v = 1.0 - std::clamp(v, 0.0, 1.0); // 翻转 V 坐标
14
15        auto i = std::clamp(static_cast<int>(u * image.width()),
16                            0, image.width() - 1);
17        auto j = std::clamp(static_cast<int>(v * image.height()),
18                            0, image.height() - 1);
19
20        auto pixel = image.data(i, j);
21        auto r = static_cast<double>(pixel[0]) / 255.0;
22        auto g = static_cast<double>(pixel[1]) / 255.0;
23        auto b = static_cast<double>(pixel[2]) / 255.0;
24
25        return Color(r, g, b);
26    }
27 };

```

2.2.3.4 Perlin 噪声纹理

Perlin 噪声用于生成自然的程序化纹理（大理石、云层等）。

```

1  class Perlin {
2      static constexpr int point_count = 256;
3      Vec3 randvec[point_count];
4      int perm_x[point_count], perm_y[point_count], perm_z[point_count];
5
6  public:
7      double noise(const Point3& p) const {
8          auto u = p.x() - std::floor(p.x());
9          auto v = p.y() - std::floor(p.y());
10         auto w = p.z() - std::floor(p.z());
11         auto i = int(std::floor(p.x())) & 255;
12         auto j = int(std::floor(p.y())) & 255;
13         auto k = int(std::floor(p.z())) & 255;
14
15         Vec3 c[2][2][2];
16         for (int di = 0; di < 2; di++) {
17             for (int dj = 0; dj < 2; dj++) {
18                 for (int dk = 0; dk < 2; dk++) {
19                     c[di][dj][dk] = randvec[
20                         perm_x[(i + di) & 255] ^
21                         perm_y[(j + dj) & 255] ^
22                         perm_z[(k + dk) & 255]
23                     ];
24                 }
25             }
26         }
27         return interpolation(c, u, v, w);
28     }
29
30     double turb(const Point3& p, int depth = 7) const {
31         auto accum = 0.0;
32         auto temp_p = p;
33         auto weight = 1.0;

```

cpp

```

34
35     for (int i = 0; i < depth; i++) {
36         accum += weight * noise(temp_p);
37         weight *= 0.5;
38         temp_p = temp_p * 2.0;
39     }
40
41     return std::fabs(accum);
42 }
43 };

```

2.2.3.5 MarbleTexture: 大理石纹理

```

1  class MarbleTexture : public Texture {
2      Perlin perlin;
3      double scale;
4      Vec3 dir;
5
6  public:
7      MarbleTexture(double scale, Vec3 direction)
8          : scale(scale), dir(direction) {}
9
10     Color value(double u, double v, const Point3& p) const override {
11         return Color(1, 1, 1) * 0.5 *
12             (1 + std::sin(scale * dot(p, dir) + 10 * perlin.turb(p)));
13     }
14 };

```

2.2.4 光源与发光材质

2.2.4.1 Light: 发光材质

```

1  class Light : public Material {
2      std::shared_ptr<Texture> tex;
3
4  public:
5      Light(const Color& color) : tex(std::make_shared<ColorTexture>(color)) {}
6
7      Color emit(const Ray& r_in, const HitResult& hit) const override {
8          if (!hit.front_face) {
9              return Color(0, 0, 0); // 背面不发光
10         }
11         return tex->value(hit.u, hit.v, hit.p);
12     }
13 };

```

关键特性:

- 只从正面发光 (front_face 检查)
- 不散射光线 (scatter 返回 false)
- 支持纹理化发光

使用示例:

```

1  auto light_material = Light::create(Color(15, 15, 15));
2  auto light = Quadrilateral::create(
3      Point3(213, 554, 227),

```

```

4   Vec3(130, 0, 0),
5   Vec3(0, 0, 105),
6   light_material
7 );

```

2.2.5 新型几何体

2.2.5.1 Quadrilateral: 四边形

四边形是构建复杂场景的基础，支持光源采样。

```

1  class Quadrilateral : public Shape2D, public Samplable {
2      double area;
3
4  public:
5      Quadrilateral(const Point3& origin, const Vec3& u, const Vec3& v,
6                     std::shared_ptr<Material> mat) : Shape2D(origin, u, v, mat) {
7          area = cross(u, v).norm();
8          set_bounding_box();
9      }
10
11     bool is_interior(double alpha, double beta, HitResult& result) const override {
12         auto unit = Interval(0, 1);
13         if (!unit.contains(alpha)) return false;
14         if (!unit.contains(beta)) return false;
15         result.u = alpha;
16         result.v = beta;
17         return true;
18     }
19
20     // 用于重要性采样
21     double pdf_value(const Point3& o, const Vec3& v) const override {
22         HitResult result;
23         Ray ray(o, v);
24         if (!hit(ray, Interval(0.001, infinity), result)) return 0.0;
25
26         auto distance_squared = result.t * result.t * v.sqrnorm();
27         auto cosine = std::fabs(dot(normal, v.normalized()));
28
29         return distance_squared / (cosine * area);
30     }
31
32     Vec3 random(const Point3& o) const override {
33         auto random_point = origin + random_double() * vec_u +
34                                 random_double() * vec_v;
35         return random_point - o;
36     }
37 };

```

2.2.5.2 Triangle: 三角形

```

1  class Triangle : public Shape2D {
2      bool is_interior(double alpha, double beta, HitResult& result) const override {
3          if (alpha + beta > 1) return false; // 重心坐标约束
4
5          auto unit = Interval(0, 1);

```

```

6         if (!unit.contains(alpha)) return false;
7         if (!unit.contains(beta)) return false;
8
9         result.u = alpha;
10        result.v = beta;
11        return true;
12    }
13 };

```

2.2.5.3 Box: 立方体

```

1  class Box : public Hittable {
2      HittableList sides;
3
4  public:
5      Box(const Point3& a, const Point3& b, std::shared_ptr<Material> mat) {
6          auto min = Point3(std::fmin(a.x(), b.x()),
7                          std::fmin(a.y(), b.y()),
8                          std::fmin(a.z(), b.z()));
9          auto max = Point3(std::fmax(a.x(), b.x()),
10                          std::fmax(a.y(), b.y()),
11                          std::fmax(a.z(), b.z()));
12
13          auto dx = Vec3(max.x() - min.x(), 0, 0);
14          auto dy = Vec3(0, max.y() - min.y(), 0);
15          auto dz = Vec3(0, 0, max.z() - min.z());
16
17          // 构建六个面
18          sides.add(make_shared<Quadrilateral>(
19              Point3(min.x(), min.y(), max.z()), dx, dy, mat)); // front
20          sides.add(make_shared<Quadrilateral>(
21              Point3(max.x(), min.y(), max.z()), -dz, dy, mat)); // right
22          // ... 其余四个面
23      }
24 };

```

2.2.6 参与介质

2.2.6.1 ConstantMedium: 恒定密度介质

用于模拟烟雾、雾气等体积效果。

```

1  class ConstantMedium : public Hittable {
2      std::shared_ptr<Hittable> boundary;
3      std::shared_ptr<Material> phase_function;
4      double neg_inv_density;
5
6  public:
7      ConstantMedium(std::shared_ptr<Hittable> b, double d, const Color& color)
8          : boundary(std::move(b)),
9            neg_inv_density(-1.0 / d),
10            phase_function(Isotropic::create(color)) {}
11
12      bool hit(const Ray& ray, Interval interval, HitResult& result) const override {
13          HitResult hit1, hit2;
14

```

```

15 // 找到进入和离开边界的两个交点
16 if (!boundary->hit(ray, Interval::universe(), hit1)) return false;
17 if (!boundary->hit(ray, Interval(hit1.t + 1e-4, infinity), hit2)) return false;
18
19 hit1.t = std::fmax(hit1.t, interval.min);
20 hit2.t = std::fmin(hit2.t, interval.max);
21
22 if (hit1.t >= hit2.t) return false;
23 if (hit1.t < 0) hit1.t = 0;
24
25 auto ray_length = ray.direction().norm();
26 auto distance = (hit2.t - hit1.t) * ray_length;
27
28 // 随机采样散射距离
29 auto hit_distance = neg_inv_density * std::log(random_double());
30
31 if (hit_distance > distance) return false;
32
33 result.t = hit1.t + hit_distance / ray_length;
34 result.p = ray.at(result.t);
35 result.normal = Vec3(1, 0, 0); // arbitrary
36 result.front_face = true;
37 result.mat = phase_function;
38
39 return true;
40 }
41 };

```

2.2.6.2 Isotropic: 各向同性材质

```

1 class Isotropic : public Material {
2     std::shared_ptr<Texture> tex;
3
4 public:
5     Isotropic(const Color& color) : tex(std::make_shared<ColorTexture>(color)) {}
6
7     bool scatter(const Ray& r_in, const HitResult& hit,
8                 ScatterResult& result) const override {
9         result.scattered = std::make_unique<SpherePDF>(); // 均匀散射
10        result.attenuation = tex->value(hit.u, hit.v, hit.p);
11        return true;
12    }
13
14    double scattering_pdf(const Ray& r_in, const HitResult& hit,
15                        const Ray& scattered) const override {
16        return 1 / (4 * pi); // 球面均匀分布
17    }
18 };

```

散射距离采样: $d = -\frac{\ln(\xi)}{\rho}$, 其中 ξ 是随机数, ρ 是密度。

2.3 第三部分: 蒙特卡洛积分与重要性采样

2.3.1 蒙特卡洛积分

渲染方程的核心是求解积分:

$$L_{o(p, \omega_o)} = L_{e(p, \omega_o)} + \int_{\Omega} f_{r(p, \omega_i, \omega_o)} L_{i(p, \omega_i)} \cos \theta_i d\omega_i$$

蒙特卡洛方法通过随机采样估计积分值：

$$\langle I \rangle = \frac{1}{N} \sum_{i=1}^N \frac{f(X_i)}{p(X_i)}$$

其中 $p(X)$ 是概率密度函数 (PDF)。

2.3.1.1 Vec3PDF 基类

```
1 class Vec3PDF {
2 public:
3     virtual ~Vec3PDF() = default;
4     virtual double value(const Vec3& direction) const = 0; // PDF值
5     virtual Vec3 generate() const = 0; // 采样方向
6 };
```

2.3.1.2 SpherePDF: 球面均匀采样

```
1 class SpherePDF : public Vec3PDF {
2 public:
3     double value(const Vec3& direction) const override {
4         return 1 / (4 * pi); // 均匀分布
5     }
6
7     Vec3 generate() const override {
8         return Vec3::random_unit();
9     }
10 };
```

2.3.2 重要性采样

重要性采样通过选择合适的 PDF 来降低方差，提高收敛速度。

2.3.2.1 CosinePDF: 余弦加权采样

用于漫反射表面，PDF 与 $\cos \theta$ 成正比。

```
1 class CosinePDF : public Vec3PDF {
2     OrthonormalBasis uvw;
3
4 public:
5     CosinePDF(const Vec3& w) : uvw(w) {}
6
7     double value(const Vec3& direction) const override {
8         auto cosine = dot(direction.normalized(), uvw.w());
9         return (cosine <= 0) ? 0.0 : (cosine / pi);
10    }
11
12    Vec3 generate() const override {
13        return uvw.transform(Vec3::random_cosine_z());
14    }
15 };
```

PDF 公式: $p(\omega) = \frac{\cos \theta}{\pi}$

采样方法 (Malley's method):

1. 在单位圆内均匀采样 (x, y)
2. 计算 $z = \sqrt{1 - x^2 - y^2}$
3. 得到球面点 (x, y, z)

```
1 static Vec3 random_cosine_z() {
2     auto r1 = random_double();
3     auto r2 = random_double();
4     auto phi = 2 * pi * r1;
5     auto x = std::cos(phi) * std::sqrt(r2);
6     auto y = std::sin(phi) * std::sqrt(r2);
7     auto z = std::sqrt(1 - r2);
8     return Vec3(x, y, z);
9 }
```

cpp

2.3.2.2 ObjectPDF: 几何体采样

直接对光源进行采样, 适用于场景中有明确光源的情况。

```
1 class ObjectPDF : public Vec3PDF {
2     const Samplable& objects;
3     Point3 origin;
4
5 public:
6     ObjectPDF(const Samplable& objects, const Point3& origin)
7         : objects(objects), origin(origin) {}
8
9     double value(const Vec3& direction) const override {
10         return objects.pdf_value(origin, direction);
11     }
12
13     Vec3 generate() const override {
14         return objects.random(origin);
15     }
16 };
```

cpp

对于四边形光源:

```
1 double pdf_value(const Point3& o, const Vec3& v) const override {
2     HitResult result;
3     if (!hit(Ray(o, v), Interval(0.001, infinity), result)) return 0.0;
4
5     auto distance_squared = result.t * result.t * v.sqrnorm();
6     auto cosine = std::fabs(dot(normal, v.normalized()));
7
8     return distance_squared / (cosine * area);
9 }
```

cpp

PDF 转换公式: $p(\omega) = p(A) \frac{r^2}{\cos \theta \cdot A}$

2.3.3 正交基 ONB

用于在局部坐标系和世界坐标系之间转换。

```
1 class OrthonormalBasis {
2     Vec3 axis[3];
```

cpp

```

3
4 public:
5     explicit OrthonormalBasis(const Vec3& n) {
6         axis[2] = n.normalized(); // w轴 (法线方向)
7         Vec3 a = (std::fabs(axis[2].x()) > 0.9) ?
8             Vec3(0, 1, 0) : Vec3(1, 0, 0);
9         axis[1] = cross(axis[2], a).normalized(); // v轴
10        axis[0] = cross(axis[2], axis[1]); // u轴
11    }
12
13    const Vec3& u() const { return axis[0]; }
14    const Vec3& v() const { return axis[1]; }
15    const Vec3& w() const { return axis[2]; }
16
17    Vec3 transform(const Vec3& a) const {
18        return a.x() * axis[0] + a.y() * axis[1] + a.z() * axis[2];
19    }
20 };

```

构建方法:

1. 法线 n 作为 w 轴
2. 选择辅助向量 a (避免与 n 平行)
3. 计算 $v = w \times a$
4. 计算 $u = w \times v$

2.3.4 混合采样

2.3.4.1 MixturePDF: 混合概率密度函数

结合多种采样策略 (如 BRDF 采样 + 光源采样), 平衡探索与利用。

```

1 class MixturePDF : public Vec3PDF {
2     std::unique_ptr<Vec3PDF> p0, p1;
3     double weight_1;
4
5 public:
6     MixturePDF(std::unique_ptr<Vec3PDF> p0, std::unique_ptr<Vec3PDF> p1,
7         double w1 = 0.5)
8         : p0(std::move(p0)), p1(std::move(p1)), weight_1(w1) {}
9
10    double value(const Vec3& direction) const override {
11        return (1 - weight_1) * p0->value(direction) +
12            weight_1 * p1->value(direction);
13    }
14
15    Vec3 generate() const override {
16        if (random_double() < weight_1) {
17            return p1->generate();
18        } else {
19            return p0->generate();
20        }
21    }
22 };

```

使用示例:

```

1 std::unique_ptr<Vec3PDF> scatter_pdf =
2     sample_target ? std::make_unique<MixturePDF>(<
3         std::make_unique<ObjectPDF>(*sample_target, hit_result.p),
4         std::move(pdf))
5     : std::move(pdf);

```

混合策略的优势:

- 降低方差: 结合多种采样方法
- 鲁棒性强: 适应不同场景配置
- 收敛更快: 充分利用场景信息

2.3.5 康奈尔盒场景

经典测试场景，用于验证光照计算的正确性。

```

1 auto cornell_box() {
2     HittableList world;
3
4     auto red = Lambertian::create(Color(.65, .05, .05));
5     auto white = Lambertian::create(Color(.73, .73, .73));
6     auto green = Lambertian::create(Color(.12, .45, .15));
7     auto light = Light::create(Color(15, 15, 15));
8
9     // 墙壁
10    world.add(Quadrilateral::create(
11        Point3(555, 0, 0), Vec3(0, 555, 0), Vec3(0, 0, 555), green));
12    world.add(Quadrilateral::create(
13        Point3(0, 0, 0), Vec3(0, 555, 0), Vec3(0, 0, 555), red));
14    world.add(Quadrilateral::create(
15        Point3(0, 0, 0), Vec3(555, 0, 0), Vec3(0, 0, 555), white));
16
17    // 顶部光源
18    world.add(Quadrilateral::create(
19        Point3(213, 554, 227), Vec3(130, 0, 0), Vec3(0, 0, 105), light));
20
21    // 盒子
22    auto box1 = Box::create(Point3(0, 0, 0), Point3(165, 330, 165), white);
23    auto box2 = Box::create(Point3(0, 0, 0), Point3(165, 165, 165), white);
24
25    return world;
26 }

```

康奈尔盒特点:

- 左红右绿墙面 (测试颜色渗透)
- 顶部区域光源
- 两个不同高度的白色立方体
- 标准尺寸: $555 \times 555 \times 555$

2.3.6 现代渲染架构

2.3.6.1 完整渲染流程

```
1 auto render(const Hittable& world, Samplable* sample_target = nullptr) { cpp
2     initialize();
3     auto image = std::vector<Color>(image_width * image_height);
4
5     std::atomic<int> counter = 0;
6     #pragma omp parallel for
7     for (int j = 0; j < image_height; ++j) {
8         for (int i = 0; i < image_width; ++i) {
9             Color pixel_color(0, 0, 0);
10            int num_samples = 0;
11
12            for (int sj = 0; sj < sqrt_spp; sj++) {
13                for (int si = 0; si < sqrt_spp; si++) {
14                    Ray r = get_ray(i, j, si, sj);
15                    auto color = ray_color(r, world, sample_target, max_depth);
16
17                    // NaN检测
18                    if (color.r() != color.r() || color.g() != color.g() ||
19                        color.b() != color.b()) {
20                        continue;
21                    }
22                    num_samples++;
23                    pixel_color += color;
24                }
25            }
26            image[j * image_width + i] = pixel_color / (double)num_samples;
27        }
28    }
29    return RenderResult{.width = image_width, .height = image_height,
30                        .data = std::move(image)};
31 }
```

2.3.6.2 智能采样调度

```
1 auto color_scatter = Match{std::move(scattered)}{ cpp
2     [&](Ray ray) -> Color {
3         // 确定性散射 (镜面反射、折射)
4         return attenuation * ray_color(ray, world, sample_target, depth - 1);
5     },
6     [&](std::unique_ptr<Vec3PDF> pdf) -> Color {
7         // 随机散射 (漫反射)
8         std::unique_ptr<Vec3PDF> scatter_pdf =
9             sample_target ? std::make_unique<MixturePDF>(
10                 std::make_unique<ObjectPDF>(*sample_target, hit_result.p),
11                 std::move(pdf))
12             : std::move(pdf);
13
14         auto scattered_ray = Ray(hit_result.p, scatter_pdf->generate(), ray.time());
15         double sampling_prob = scatter_pdf->value(scattered_ray.direction());
16         double scatter_prob =
17             hit_result.mat->scattering_pdf(ray, hit_result, scattered_ray);
18
19         Color sampled_color = ray_color(scattered_ray, world, sample_target, depth - 1);
20         return (attenuation * scatter_prob * sampled_color) / sampling_prob;
21     }
22 );
```

核心优化技术:

1. **OpenMP 并行化**: #pragma omp parallel for
2. 分层采样: 减少方差
3. 混合重要性采样: BRDF + 光源采样
4. **BVH 加速**: $O(\log N)$ 光线求交
5. **NaN 检测**: 避免数值错误
6. 自适应深度: Russian Roulette (可选)

渲染方程的最终蒙特卡洛估计:

$$L_o \approx L_e + \frac{f_r \cdot p_{\text{scatter}} \cdot L_i}{p_{\text{sample}}}$$

其中:

- f_r : BRDF
- p_{scatter} : 散射概率
- p_{sample} : 采样概率
- L_i : 入射辉度 (递归计算)

2.4 第四部分: 光谱渲染 (Spectral Rendering)

2.4.1 概述与动机

传统的 RGB 渲染虽然计算简单, 但存在物理准确性问题: RGB 只是人眼对可见光谱的简化表示, 无法精确模拟真实光线的波长相关行为。光谱渲染通过在整个可见光波长范围内进行计算, 提供了更物理准确的光照模拟。

光谱渲染的优势:

1. 物理准确性: 直接模拟光的波长特性
2. 色散模拟: 棱镜等色散现象 (通过波长相关折射率)
3. 波长采样: 材质属性可以在特定波长处采样
4. 色彩管理: 更准确的色彩空间转换 (通过 CIE XYZ)

核心实现 **commit** 参考:

- ca89419: 光谱数据结构与 CIE 色彩匹配函数
- 47d5f67: RGB 到光谱转换 (Smits 1999 方法)
- 336d96f: 材质系统重构, 支持波长相关的材质属性

2.4.2 重构概览: 从 RGB 到 Spectrum

之前的 RGB 架构:

```
1 // 旧实现: 直接使用 Vec3 表示颜色
2 class Material {
3     Color albedo; // Color = Vec3, 三通道 RGB
4     virtual Color scatter(...) const;
5 };
```

重构后的 Spectrum 架构:

```
1 // 新实现: 使用 Spectrum 类进行光谱采样
2 class Material {
3     Spectrum albedo; // 60个波长样本 (400-700nm)
```

```
4     virtual Spectrum scatter(...) const;
5 };
```

关键变化:

1. 颜色表示: Vec3 (RGB) $\rightarrow$ Spectrum (60 samples)
2. 波长范围: 400nm - 700nm (可见光)
3. 采样策略: 均匀分布 60 个样本点
4. 转换接口: 支持 RGB $\leftrightarrow$ Spectrum 双向转换

2.4.3 Spectrum 类实现

2.4.3.1 基础结构

```
1 class Spectrum {
2 public:
3     static constexpr int nSamples = 60;           // 采样数量
4     static constexpr double lambdaStart = 400;    // 起始波长 (nm)
5     static constexpr double lambdaEnd = 700;      // 结束波长 (nm)
6
7 private:
8     std::array<double, nSamples> c = {};          // 光谱样本值
9
10    // 静态数据: CIE XYZ 匹配函数、RGB 转换表
11    struct StaticData {
12        std::array<double, nSamples> sX, sY, sZ;   // CIE 色彩匹配
13        // RGB to Spectrum 转换表 (反射 + 发光)
14        std::array<double, nSamples> rgbRefl2Spect[7];
15        std::array<double, nSamples> rgbIllum2Spect[7];
16        bool initialized = false;
17    };
18 };
```

设计要点:

- 60 个样本覆盖 400-700nm 可见光范围
- 每个样本间隔约 5nm
- 采用懒加载: 首次使用时初始化转换表

2.4.3.2 RGB 到光谱转换 (Smits 1999)

核心算法基于 **Smits 1999** 论文, 使用基函数插值:

```
1 static Spectrum fromRGB(const double rgb[3], SpectrumType type) {
2     auto& white = (type == Reflectance) ? rgbRefl2SpectWhite : rgbIllum2SpectWhite;
3     auto& cyan = (type == Reflectance) ? rgbRefl2SpectCyan : rgbIllum2SpectCyan;
4     auto& magenta = /* ... */;
5     auto& yellow = /* ... */;
6     auto& red = /* ... */;
7     auto& green = /* ... */;
8     auto& blue = /* ... */;
9
10    Spectrum result;
11
12    // 根据 RGB 大小关系选择插值路径
13    if (rgb[0] <= rgb[1] && rgb[0] <= rgb[2]) {
14        // R 最小: white + cyan + (blue or green)
15        result += rgb[0] * white;
16        if (rgb[1] <= rgb[2]) {
17            result += (rgb[1] - rgb[0]) * cyan;
18            result += (rgb[2] - rgb[1]) * blue;
```

```

19     } else {
20         result += (rgb[2] - rgb[0]) * cyan;
21         result += (rgb[1] - rgb[2]) * green;
22     }
23 }
24 // ... 其他分支 (G 最小, B 最小)
25
26 return result.clamp();
27 }

```

算法原理:

1. RGB 空间被分解为 6 个四面体区域
2. 每个区域用 3 个基函数插值
3. 基函数: 白、青、品红、黄、红、绿、蓝
4. 区分 反射光谱 和 发光光谱 两种类型

类型区分:

- SpectrumType::Reflectance: 表面反射率 (0-1)
- SpectrumType::Illuminant: 光源发光强度 (可 > 1)

2.4.3.3 光谱到 RGB 转换

通过 CIE XYZ 色彩匹配函数进行转换:

```

1 void toXYZ(double xyz[3]) const {
2     xyz[0] = xyz[1] = xyz[2] = 0;
3     for (int i = 0; i < nSamples; ++i) {
4         xyz[0] += sX[i] * c[i]; // X 通道
5         xyz[1] += sY[i] * c[i]; // Y 通道 (亮度)
6         xyz[2] += sZ[i] * c[i]; // Z 通道
7     }
8     // 归一化
9     double scale = (lambdaEnd - lambdaStart) / (CIE_Y_integral * nSamples);
10    xyz[0] *= scale; xyz[1] *= scale; xyz[2] *= scale;
11 }
12
13 void toRGB(double rgb[3]) const {
14     double xyz[3];
15     toXYZ(xyz);
16     XYZToRGB(xyz, rgb); // 标准 XYZ -> RGB 矩阵变换
17 }

```

转换流程:

```

1 Spectrum (60 samples)
2   ↓ 与 CIE XYZ 匹配函数卷积
3 XYZ 三刺激值
4   ↓ 线性变换矩阵
5 RGB 颜色

```

CIE XYZ 色彩匹配函数:

- 基于人眼视锥细胞响应曲线
- 81 个标准样本点 (380-780nm, 5nm 间隔)
- 通过插值映射到 60 个光谱样本

2.4.4 CIE 色彩匹配与转换

2.4.4.1 CIE 1931 标准观察者

```
1 namespace spectrum_data {
2     // CIE XYZ 匹配函数 (380–780nm, 81 个样本)
3     inline constexpr int nCIESamples = 81;
4     inline constexpr double CIE_lambda[nCIESamples] = {
5         380, 385, 390, ..., 775, 780
6     };
7     inline constexpr double CIE_X[nCIESamples] = { /* ... */ };
8     inline constexpr double CIE_Y[nCIESamples] = { /* ... */ };
9     inline constexpr double CIE_Z[nCIESamples] = { /* ... */ };
10    inline constexpr double CIE_Y_integral = 106.856895;
11 }
```

功能:

- $\bar{x}(\lambda), \bar{y}(\lambda), \bar{z}(\lambda)$: 三刺激值函数
- 将任意光谱分布转换为 XYZ 颜色空间
- $\bar{y}(\lambda)$ 对应人眼亮度响应 ($V(\lambda)$ 函数)

2.4.4.2 XYZ ↔ RGB 转换矩阵

```
1 // XYZ → sRGB (D65 白点)
2 inline void XYZToRGB(const double xyz[3], double rgb[3]) {
3     rgb[0] = 3.240479 * xyz[0] - 1.537150 * xyz[1] - 0.498535 * xyz[2];
4     rgb[1] = -0.969256 * xyz[0] + 1.875991 * xyz[1] + 0.041556 * xyz[2];
5     rgb[2] = 0.055648 * xyz[0] - 0.204043 * xyz[1] + 1.057311 * xyz[2];
6 }
7
8 // sRGB → XYZ
9 inline void RGBToXYZ(const double rgb[3], double xyz[3]) {
10    xyz[0] = 0.412453 * rgb[0] + 0.357580 * rgb[1] + 0.180423 * rgb[2];
11    xyz[1] = 0.212671 * rgb[0] + 0.715160 * rgb[1] + 0.072169 * rgb[2];
12    xyz[2] = 0.019334 * rgb[0] + 0.119193 * rgb[1] + 0.950227 * rgb[2];
13 }
```

标准:

- sRGB 色彩空间 (IEC 61966-2-1)
- D65 标准光源 (色温 6500K)
- 线性 RGB (未应用 gamma 校正)

2.4.5 光谱采样与插值

2.4.5.1 波长采样

```
1 double operator()(double lambda) const {
2     if (lambda < lambdaStart || lambda > lambdaEnd) return 0;
3
4     // 线性插值
5     double t = (lambda - lambdaStart) / (lambdaEnd - lambdaStart) * nSamples;
6     int i = std::min(int(t), nSamples - 1);
7     double dt = t - i;
8
9     if (i >= nSamples - 1) return c[nSamples - 1];
10    return (1 - dt) * c[i] + dt * c[i + 1];
11 }
```

用途:

- 查询任意波长的光谱值

- 支持色散模拟（不同波长不同折射率）

2.4.5.2 区间平均采样

```
1 inline double averageSpectrumSamples(
2     const double* lambda, const double* vals, int n,
3     double lambdaStart, double lambdaEnd) {
4
5     // 梯形积分法
6     double sum = 0;
7     for (int i = 0; i + 1 < n && lambdaEnd >= lambda[i]; ++i) {
8         double segStart = std::max(lambdaStart, lambda[i]);
9         double segEnd = std::min(lambdaEnd, lambda[i + 1]);
10        sum += 0.5 * (interp(segStart, i) + interp(segEnd, i))
11            * (segEnd - segStart);
12    }
13    return sum / (lambdaEnd - lambdaStart);
14 }
```

应用场景：

- 初始化时，将 CIE 81 样本映射到 60 样本
- 保证能量守恒的降采样

2.4.6 光谱运算

```
1 class Spectrum {
2     // 算术运算（逐波长）
3     Spectrum operator+(const Spectrum& s) const; // 光谱叠加
4     Spectrum operator*(const Spectrum& s) const; // 逐波长相乘（光谱调制）
5     Spectrum operator*(double a) const; // 标量缩放
6
7     // 工具函数
8     double y() const; // 计算亮度（Y 通道）
9     double maxComponent() const; // 最大光谱值
10    bool isBlack() const; // 是否为黑色
11    Spectrum clamp(double low = 0, double high = 1e30) const;
12
13    // 数学函数（逐波长应用）
14    friend Spectrum sqrt(const Spectrum& s);
15    friend Spectrum exp(const Spectrum& s);
16    friend Spectrum pow(const Spectrum& s, double e);
17    friend Spectrum lerp(double t, const Spectrum& s1, const Spectrum& s2);
18 };
```

物理意义：

- $a * b$ ：反射率调制（材质反射率 $\times$ 入射光）
- $a + b$ ：光谱能量叠加（多光源）
- $\text{sqrt}(s)$ ：用于重要性采样的方差计算

2.4.7 材质系统重构

2.4.7.1 波长相关的材质属性

实际实现中，光谱渲染主要用于两个方面：

1. 金属材质的波长采样：存储完整光谱，在特定波长处采样
2. 电介质的色散模拟：折射率随波长变化（Cauchy 方程）

```
1 class Metal : public Material {
2     Spectrum albedo; // 光谱反射率
```

```

3     double fuzz;
4
5 public:
6     Metal(const Color& color, double fuzz = 0) : albedo(color), fuzz(fuzz) {}
7
8     bool scatter(const Ray& r_in, const HitResult& hit,
9                  ScatterResult& result) const override {
10         Vec3 reflected = reflect(r_in.direction().normalized(), hit.normal).normalized() +
11             (fuzz * Vec3::random_unit());
12         result.scattered = r_in.redirect(hit.p, reflected);
13         // 在光线的波长处采样反射率
14         result.attenuation = albedo.value(r_in.wavelength());
15         return dot(reflected, hit.normal) > 0;
16     }
17 };

```

2.4.7.2 示例：色散材质

```

1 class Dielectric : public Material {
2     double base_index; // 基础折射率 (Cauchy 方程中的 A)
3     double dispersion; // 色散系数 (Cauchy 方程中的 B, 单位:  $\mu\text{m}^2$ )
4
5 public:
6     // Cauchy 方程:  $n = A + B/\lambda^2$ 
7     // 玻璃:  $A \approx 1.5$ ,  $B \approx 0.004\text{--}0.01 \mu\text{m}^2$ 
8     // 钻石:  $A \approx 2.4$ ,  $B \approx 0.01\text{--}0.02 \mu\text{m}^2$ 
9     Dielectric(double ri, double dispersion_coeff = 0.3)
10         : base_index(ri), dispersion(dispersion_coeff) {}
11
12     // 使用 Cauchy 方程计算给定波长的折射率
13     double refractive_index(double wavelength_nm) const {
14         if (dispersion == 0.0) return base_index;
15         double lambda_um = wavelength_nm / 1000.0; // nm  $\rightarrow \mu\text{m}$ 
16         return base_index + dispersion / (lambda_um * lambda_um);
17     }
18
19     bool scatter(const Ray& r_in, const HitResult& hit,
20                  ScatterResult& result) const override {
21         result.attenuation = 1.0;
22         // 根据光线波长计算折射率
23         double ri = refractive_index(r_in.wavelength());
24         // ... 折射/反射逻辑
25     }
26 };

```

核心特性:

- 金属: 完整光谱存储, 波长处采样 (`albedo.value(wavelength)`)
- 电介质: Cauchy 方程实现色散 ($n = A + B/\lambda^2$)
- 光线追踪: 每条光线携带波长信息 (`Ray.wavelength()`)
- 颜色转换: 最终通过 CIE XYZ 转换回 RGB

2.4.8 向后兼容性

为保证平滑过渡, Spectrum 类提供 RGB 兼容接口:

```

1 class Spectrum {
2     // 从 Color/Vec3 隐式构造
3     explicit Spectrum(const Color& color);
4     explicit Spectrum(const Vec3& rgb);

```

```

5
6    // 转换回 Color
7    Color toColor() const;
8
9    // 静态工厂函数
10   static Spectrum fromRGB(const Color& color,
11                           SpectrumType type = SpectrumType::Reflectance);
12
13   // 旧 API 别名
14   static constexpr double visible_min = lambdaStart;
15   static constexpr double visible_max = lambdaEnd;
16   static Spectrum constant(double v) { return Spectrum(v); }
17 };

```

使用示例：

```

1 // 旧代码仍可运行
2 Material* mat = new Lambertian(Color(0.8, 0.3, 0.3));
3
4 // 内部自动转换：
5 // Color → Spectrum::fromRGB(...) → 60 个波长样本
6
7 // 渲染输出时：
8 Spectrum pixel_spectrum = trace_ray(...);
9 Color rgb = pixel_spectrum.toColor(); // Spectrum → RGB

```

2.4.9 性能考量

内存占用：

- RGB: $3 \times 8 = 24$ 字节
- Spectrum: $60 \times 8 = 480$ 字节
- 增幅：20 倍

计算开销：

- RGB 运算：3 次浮点操作
- Spectrum 运算：60 次浮点操作
- 增幅：20 倍

优化策略：

1. 惰性初始化：转换表仅初始化一次

适用场景：

- ✓ 需要色散效果的场景（棱镜）
- ✓ 需要物理准确波长模拟的科学可视化
- ✓ 宝石等具有强色散材质的渲染
- ✗ 实时渲染（性能开销大）
- ✗ 简单场景（RGB 已足够）

2.4.10 扩展应用

2.4.10.1 色散模拟（Cauchy 方程）

```

1 class Dielectric : public Material {
2     double base_index; // 基础折射率 A
3     double dispersion; // 色散系数 B (单位:  $\mu\text{m}^2$ )
4
5     // 波长相关折射率 (Cauchy 方程)
6     double refractive_index(double wavelength_nm) const {

```

```
7         if (dispersion == 0.0) return base_index;
8         double lambda_um = wavelength_nm / 1000.0; // nm → μm
9         return base_index + dispersion / (lambda_um * lambda_um);
10    }
11 };
```

实际应用：

- 玻璃、水晶的色散效果
- 棱镜分光现象
- 不同波长的不同折射路径

2.4.11 小结

光谱渲染重构实现了从 RGB 到物理准确光谱的跨越：

架构变化：

- 核心颜色表示：Vec3 → Spectrum (60 samples)
- 转换系统：Smits 1999 RGB ↔ Spectrum
- 色彩科学：CIE XYZ 色彩匹配函数
- 材质系统：支持波长相关的材质属性

技术亮点：

- 物理准确的可见光建模（400-700nm）
- 能量守恒的转换算法
- 完整的向后兼容性
- 支持色散波长相关效果

实现参考：

- PBRT-v3 光谱系统设计
- Physically Based Rendering 4th Edition
- Smits 1999: “An RGB-to-Spectrum Conversion for Reflectances”

2.5 总结

本光线追踪项目实现了从基础到高级的完整渲染管线：

第一部分 建立了核心框架：

- Vec3 向量数学库
- 光线追踪基础（Ray, Camera）
- 几何求交（Sphere, Hittable）
- 物理材质（Lambertian, Metal, Dielectric）
- 递归路径追踪与景深效果

第二部分 引入优化与扩展：

- 运动模糊（时间维度）
- BVH 空间加速（对数时间复杂度）
- 纹理系统（图像、Perlin 噪声）
- 面光源（Light 材质）
- 复杂几何体（Quad, Triangle, Box）
- 体积渲染（ConstantMedium）

第三部分 实现高级采样技术：

- 蒙特卡洛积分理论
- 重要性采样 (CosinePDF, ObjectPDF)
- 正交基坐标变换
- 混合采样策略 (MixturePDF)
- 康奈尔盒经典场景

第四部分 光谱渲染系统:

- 从 RGB 到光谱的架构重构 (60 个波长样本)
- Smits 1999 RGB $\leftrightarrow$ Spectrum 转换算法
- CIE XYZ 色彩匹配与标准观察者
- 波长相关的材质属性 (色散、波长采样)
- 支持色散物理效果

渲染器性能指标:

- 支持数万级物体场景 (BVH 加速)
- 多线程并行渲染 (OpenMP)
- 物理正确的光线传输
- 灵活的材质与纹理系统

未来改进方向:

- 双向路径追踪 (BDPT)
- 光子映射 (Photon Mapping)
- 梯度域渲染 (Gradient-domain)
- GPU 加速 (CUDA/OptiX)
- 实时降噪 (AI Denoiser)

三、 总结与体会

这次实验是关于光线追踪 (Ray Tracing) 的实现, 是计算机图形学中一个经典且重要的主题。通过实现一个完整的光线追踪渲染器, 我深入理解了光线在场景中的传播、材质与光照的交互, 以及各种优化技术。

光线追踪的核心思想是逆向追踪光线, 从相机出发, 通过像素点发射光线, 计算光线与场景中物体的交点, 递归地追踪反射、折射等光线路径, 最终计算出像素的颜色。这种方法能够产生非常真实的渲染效果, 包括阴影、反射、折射、景深等物理现象。

在实现过程中, 我遇到了不少挑战。首先是光线与各种几何体的求交计算, 需要仔细处理数学公式和边界情况。其次是材质系统的设计, 需要支持漫反射、镜面反射、折射等多种材质类型, 并且要正确计算概率分布函数 (PDF) 以实现重要性采样。BVH (Bounding Volume Hierarchy) 加速结构的实现也很关键, 它能够大幅提升渲染性能, 特别是在处理复杂场景时。

此外, 我还实现了多种高级特性, 如纹理映射、柏林噪声、蒙特卡洛和重要性采样、光谱渲染等。这些特性不仅增强了渲染器的功能, 也让我对图形学的各个方面有了更全面的认识。

整体来说, 这次实验让我从理论到实践全面理解了光线追踪算法。虽然实现过程比较复杂, 但看到最终渲染出的真实感图像, 还是很有成就感的。这个项目不仅巩固了我的 C++ 编程能力, 也培养了我解决复杂问题的能力和耐心。